



EDA-Ordinario - Examen Ordinario Enero EDA

Estructuras de Datos Avanzadas (Universidad Rey Juan Carlos)



Scan to open on Studocu



Grado en Ingeniería Informática

Estructuras de Datos Avanzadas

Examen Convocatoria Ordinaria

15 de enero de 2025

Nombre y apellidos	
DNI / Pasaporte	

Tiempo máximo: 2 horas

Instrucciones:

- La duración del examen es de 120 minutos.
- La puntuación de cada ejercicio se indica junto al enunciado.
- No se responderán preguntas sobre los enunciados. La interpretación de estos forma parte del examen.
- No está permitido el uso de calculadoras, ni la consulta de libros, apuntes o dispositivos electrónicos de ningún tipo durante el examen.
- Desde el Aula Virtual hay que descargar un archivo ZIP que contiene el esqueleto de los ejercicios del examen y las pruebas unitarias asociadas.
- El estudiante debe generar un fichero ZIP con el directorio en el que esté el código desarrollado durante el examen y entregarlo en la actividad "Examen práctico" disponible en Aula Virtual.

Consideraciones adicionales:

- No se pueden añadir nuevas clases, métodos o propiedades públicas.
- Solo se pueden añadir nuevas clases si son privadas.
- A las clases ya dadas solo se les puede añadir métodos y propiedades con visibilidad privada.
- No se pueden modificar las cabeceras de los métodos ya dados.
- El código del proyecto proporcionado se ha probado utilizando **Oracle OpenJDK 22.0.1** (disponible en el Escritorio de Desarrollo de MyApps) y **JUnit 5.8.1**.

Ejercicio 1 [4 puntos]

Se pide implementar 3 métodos que se encuentran en la clase *AdditionalFeatures*, en el paquete *graphs*. Los métodos a implementar son los siguientes:

Método	Descripción	Puntos
<pre>public boolean isTree(Graph<V, E> g)</pre>	Recibe un grafo y devuelve <i>true</i> si el grafo es un árbol. Un grafo es un árbol cuando es conexo y el número de aristas es igual al número de nodos menos 1. Si el grafo es nulo o está vacío, devuelve <i>true</i> .	2
<pre>public Collection<Vertex<V>> sourceVertex(DirectedGraph<V, E> d)</pre>	Recibe un grafo dirigido y devuelve todos los vértices fuente de dicho grafo. Un vértice es fuente si solo tiene aristas de salida. Debe tener al menos una arista de salida.	1
<pre>public boolean isComplete(Graph<V, E> g)</pre>	Recibe un grafo y devuelve <i>true</i> si el grafo es completo. En caso contrario, devuelve <i>false</i> . Un grafo es completo si existe una arista entre cada par de nodos. Si el grafo es nulo o está vacío, no es completo.	1

Ejercicio 2 [6 puntos]

La federación española de ajedrez desea tener un sistema automatizado para organizar la clasificación (*ranking*) de sus jugadores. Cada jugador se puede identificar por su nombre, apellido y número de puntos. Los puntos de un jugador aumentan cuando gana partidas y disminuye cuando pierde. Se desea crear una clase (*PlayersRanking*) que permita guardar y operar sobre una colección de jugadores de ajedrez. La clase *Player* se facilita implementada y no debe modificarse.

En primer lugar, es necesario implementar la clase *NameComparator*. En particular, se pide implementar el siguiente método:

Método	Descripción	Puntos
public int compare(Player player1, Player player2)	Recibe dos jugadores (<i>player1</i> y <i>player2</i>). utilizando el orden alfabético, devuelve un número positivo si <i>player1</i> debe ir antes que <i>player2</i> , un número negativo si <i>player2</i> tiene que ir antes que <i>player1</i> o 0 (cero) si tienen el mismo nombre y apellido. Nótese que el orden se establece teniendo en cuenta primero el apellido y después el nombre.	0,5

A continuación, es necesario implementar la clase *PlayersRanking*. **La elección de las estructuras de datos adecuadas para resolver el ejercicio se puntuará con 0,5 puntos.** Además, deben implementarse las siguientes operaciones de la forma más eficiente posible:

Método	Descripción	Puntos
public boolean addNewPlayer(Player p)	Recibe un jugador y lo añade al <i>ranking</i> . Devuelve <i>true</i> si se ha añadido. En caso contrario, devuelve <i>false</i> . La complejidad de este método no puede ser superior a $O(\log n)$, siendo n el número de jugadores del <i>ranking</i> .	1
public boolean removePlayer(Player p)	Recibe un jugador y lo elimina del <i>ranking</i> . Devuelve <i>true</i> si se ha eliminado el jugador. En caso contrario, devuelve <i>false</i> .	1
public Iterable<Player> playersWithRanking(Integer r)	Recibe un número de puntos y devuelve un objeto <i>Iterable</i> con los jugadores del <i>ranking</i> que tienen ese número de puntos, ordenados alfabéticamente, tal y como se describe en la clase <i>NameComparator</i> .	1
public boolean modificationRankingPlayer(Player p, Integer diferencia)	Recibe un jugador y un número de puntos (puede ser un valor positivo o negativo). Actualiza los puntos del jugador con la diferencia especificada. Si el jugador existe y se han actualizado los puntos del ranking, devuelve <i>true</i> . En caso contrario, devuelve <i>false</i> .	1

<pre>public Iterable<Player> allPlayers()</pre>	Devuelve un objeto <i>Iterable</i> con todos los jugadores del <i>ranking</i>, ordenados de mayor a menor puntuación. En caso de empate, están ordenados alfabéticamente, tal y como se describe en la clase <i>NameComparator</i>. En este caso, se permite devolver un objeto de cualquier clase que implemente <i>Iterable</i>, aunque no sea la manera más eficiente de resolver el problema, tal y como hemos visto en clase. Por lo tanto, es posible devolver una lista (un objeto de tipo <i>List</i>). Por ejemplo, la siguiente línea sería válida: <pre>return new ArrayList<>(0);</pre> En caso de querer hacerlo de la manera adecuada, podéis utilizar las clases privadas dadas más abajo (<i>PlayersIterable</i> y <i>PlayersIterator</i>). No obstante, no se penalizará crear y devolver una lista con los jugadores.	1
---	--	---

Por supuesto, todos los métodos deben operar con la máxima eficiencia posible, dado que la federación tiene muchos jugadores inscritos.